

ECOM EXPERTS

Mensajero shopify

Manual de usuario, tecnico y de desarrollo

mensajero-shopify · 08/09/2026

Contenido

1. Qué hace

2. Requisitos

3. Instalación

4. Configuración inicial

 Estadísticas

 Mensajes

 Forma de pago

 Ajustes

5. Flujo típico el primer día

6. Variables disponibles por evento

7. Datos personales y RGPD

8. Resolución de problemas

El WhatsApp no llega tras un pedido

"Enviar prueba" no hace nada

Las estadísticas no cargan

9. Cancelación

10. Soporte

Estructura

Despliegue por FTP (igual que Mensajero)

App en el Partner Dashboard

Flujo implementado

Pendiente (siguientes fases)

1. Arquitectura general

2. Componentes y ficheros

3. Modelo de datos (3 tablas)

4. Flujos completos

4.1 Instalación (OAuth)

4.2 Webhook entrante (pedido nuevo)

4.3 UI embebida (App Bridge)

4.4 Acciones autenticadas por session token

5. Seguridad

6. Dependencias

7. Variables de entorno (config/config.php)

1. Setup local

2. Despliegue de configuración (Shopify CLI)

3. Estructura de código

app/Router.php

app/Shopify.php

app/Events.php

app/Mensajero.php

app/Db.php

app/Crypto.php

routes.php

4. Depuración

Log de la app

Logs PHP

Comprobaciones útiles

5. Añadir una pestaña a la UI

6. Migraciones de BD

7. Compatibilidad PHP

8. Pruebas manuales

9. Comandos útiles

1. Resumen ejecutivo

2. Por qué necesitas una app NUEVA (no esta)

3. Cambios técnicos necesarios en el código

3.1 Token Exchange (OBLIGATORIO en modelo público)

3.2 Mantener OAuth clásico como fallback (opcional)

3.3 Pantalla de elección de plan tras la instalación (opcional)

4. Trabajo no técnico (Partner Dashboard / Dev Dashboard)

5. Revisión de Shopify

6. Facturación

Opción A (recomendada): cobras tú en Mensajero

Opción B: Shopify Billing API

7. Soporte multi-tienda por cliente

8. Roadmap concreto

Semana 1

Semana 2

Tras aprobación

9. Riesgos y mitigación

10. Coste estimado de esta migración

Envía avisos automáticos por WhatsApp a tus clientes cuando hacen un pedido en tu tienda Shopify (confirmación, envío, reembolso, bienvenida).

1. Qué hace

Conecta tu Shopify con tu cuenta de Mensajero. Cada vez que ocurre algo importante en un pedido, te enviamos por WhatsApp el mensaje que tú definas.

Eventos disponibles:

-  **Pedido nuevo** – al recibir el pedido (confirmación al cliente).
-  **Pedido enviado (con seguimiento)** – cuando marcas el pedido como preparado/fulfilled. Incluye número y URL de tracking.
-  **Reembolso** – cuando emites un reembolso.
-  **Bienvenida** – cuando un cliente nuevo crea su cuenta en tu tienda.

Cada evento permite **mensajes diferentes por idioma** (Español, Inglés, Francés, Alemán, Italiano, Portugués). Si el pedido viene de un cliente cuyo idioma no tienes traducido, se usa el "Por defecto".

2. Requisitos

1. Una tienda Shopify activa.
2. Una cuenta en **mensajero.gmartos.es** con plan contratado.
3. WhatsApp conectado en tu cuenta de Mensajero (escaneo del QR).
4. Una **API key** generada en Mensajero (apartado API keys).

3. Instalación

1. Tu proveedor te dará un enlace de instalación. Ábrelo desde el admin de tu Shopify.
2. Acepta los permisos solicitados (leer pedidos, clientes, envíos).
3. Shopify te devolverá automáticamente al admin con la app abierta.

4. Configuración inicial

La app tiene 4 pestañas:

Estadísticas

Tu cuota mensual, envíos del día, estado de la conexión de WhatsApp y los últimos mensajes enviados. Esta pestaña se llena cuando ya tienes la API key configurada.

Mensajes

Una tarjeta por evento. Para activar un aviso:

1. Pulsa el **interruptor** de la cabecera de la tarjeta (a la derecha).
2. Edita el mensaje en la pestaña **"Por defecto"**. Usa las variables: `{firstname}`, `{order}`, `{total}`, `{products}`, `{tracking}`, `{tracking_url}`, `{shop}`.
3. Si quieres, traduce a otros idiomas en las pestañas **Español / English / Français /** Si dejas un idioma vacío, ese cliente recibirá el "Por defecto".
4. Pulsa **Guardar** (botón verde de la barra inferior).

Forma de pago

Permite excluir pedidos de marketplaces (Amazon, Temu, Mirakl...) que no quieres notificar.

- **Modo:**
- **Enviar a todos** – sin filtro.
- **Enviar SOLO a los de la lista** – solo a los que aparezcan en la lista.
- **Excluir los de la lista** – todos menos los de la lista.
- **Lista:** separa con comas las pasarelas/canales que quieres incluir o excluir. La app compara contra el nombre del canal y la pasarela del pedido (Shopify usa `source_name`, `gateway`, `payment_gateway_names`).

Ejemplo: para no enviar a pedidos de Amazon: - Modo: **Excluir los de la lista** - Lista: `amazon`

Ajustes

- **API key de Mensajero:** pégala aquí. Una vez guardada, el campo aparece como `.....` y dejarlo en blanco mantiene la guardada.
- **Enviar prueba:** introduce un teléfono con prefijo (ej. `34600111222`) y pulsa "Enviar prueba". Debe llegar el WhatsApp al instante. Si ves "ip_not_allowed", pulsa **"Autorizar este servidor"** para añadir la IP a tu lista blanca en Mensajero.
- **Conectar WhatsApp:** enlace a tu panel de Mensajero para escanear el QR. Esto NO se hace en Shopify, por seguridad.
- **Diagnóstico** (al final): si los avisos no llegan, **Comprobar webhooks y log** te dice si Shopify está mandando los eventos.

5. Flujo típico el primer día

1. Instalar la app.
2. Ir a **Ajustes** → pegar API key → **Guardar**.
3. **Autorizar este servidor** para que tu cuenta Mensajero acepte envíos desde este servidor.
4. **Enviar prueba** con tu teléfono → debe llegar el WhatsApp.
5. Ir a **Mensajes** → encender el interruptor de "Pedido nuevo" → personalizar el texto si quieres → **Guardar**.
6. Hacer un pedido de prueba en tu tienda (con un cliente que tenga teléfono) → debe llegar el aviso.
7. Encender el resto de eventos según lo que quieras notificar.

6. Variables disponibles por evento

Evento	Variables
Pedido nuevo	<code>{firstname}</code> <code>{order}</code> <code>{total}</code> <code>{products}</code> <code>{shop}</code>
Pedido enviado	<code>{firstname}</code> <code>{order}</code> <code>{tracking}</code> <code>{tracking_url}</code> <code>{shop}</code>
Reembolso	<code>{firstname}</code> <code>{order}</code> <code>{shop}</code>
Bienvenida	<code>{firstname}</code> <code>{shop}</code>

7. Datos personales y RGPD

- La app trata únicamente **nombre y teléfono** de tus clientes para entregar el mensaje.
- Los datos se transmiten cifrados a Mensajero (servidores en Alemania, UE), donde se conservan un **máximo de 2 días** y luego se anonimizan.
- Debes **firmar el Contrato de Encargado del Tratamiento (DPA)** en tu cuenta de Mensajero (`mensajero.gmartos.es/account/dpa`). La app te avisará con un banner rojo arriba si no lo has firmado.
- Si un cliente solicita borrado de sus datos, puedes anonimizarlos al instante desde Mensajero (`Borrado real`).

8. Resolución de problemas

El WhatsApp no llega tras un pedido

1. Ve a  **Ajustes** → **Diagnóstico** → **Comprobar webhooks y log**.
2. Revisa que entre los topics registrados aparezcan `orders/create`, `fulfillments/create`, etc.
3. Si en el log no hay líneas tras tu acción, el webhook no llegó (problema de instalación: contacta soporte).
4. Si en el log hay una línea pero con `err=missing_to_or_text` o `sin telefono valido`, el cliente del pedido **no tiene teléfono con prefijo**. Edita el cliente en Shopify y añádeselo.
5. Si dice `ip_not_allowed`, ve a Ajustes → **Autorizar este servidor**.

"Enviar prueba" no hace nada

Estás abriendo la app por URL directa en vez de embebida. Cierra y vuelve a abrirla desde **Apps** → **mensajero_whatshap** en el admin de Shopify.

Las estadísticas no cargan

Falta la API key (configúrala en Ajustes) o no hay envíos todavía.

9. Cancelación

- Para dejar de recibir avisos: apaga los interruptores en Mensajes y guarda.
- Para desinstalar: desde el admin de Shopify, Apps → mensajero_whatshap → Eliminar. Eso disparará `app/uninstalled` y borraremos automáticamente los datos de tu tienda en nuestra app.
- Para cancelar tu plan en Mensajero: panel `mensajero.gmartos.es` → Suscripción → Cancelar.

10. Soporte

- Mensajero soporte: soporte@gmartos.es
- Documentación API: `mensajero.gmartos.es/docs/api`

Mensajero Shopify (conector en PHP plano, sin Composer)

App de Shopify que escucha webhooks de la tienda y envía avisos por WhatsApp llamando a la API de **Mensajero** (el mismo backend que el módulo de PrestaShop). Hecha en **PHP nativo** para desplegar por **FTP** en tu subdominio.

Estado: **esqueleto/MVP inicial**. El OAuth y los webhooks necesitan tu **API key/secret** del Partner Dashboard para probarse de verdad.

Estructura

```
mensajero-shopify/  
├─ public/           ← DOCROOT del subdominio (index.php + .htaccess)  
├─ config/config.php ← copiar de config.sample.php y rellenar (FUERA de public)  
├─ app/             ← Shopify.php, Db.php, Mensajero.php, Router.php, helpers.php  
├─ routes.php       ← rutas (landing, OAuth, webhooks, ajustes)  
├─ views/           ← home.php (presentación), app.php (ajustes)  
├─ db/schema.sql    ← tablas shops / shop_config / webhook_seen  
└─ storage/logs/    ← logs (se crea solo)
```

Despliegue por FTP (igual que Mensajero)

1. Sube la carpeta al subdominio. **Docroot del dominio** → `.../public_html/public` (config/ y app/ quedan FUERA del docroot, no accesibles por web).
2. Crea **BD + usuario** e importa `db/schema.sql`.
3. Copia `config/config.sample.php` a `config/config.php` y rellena: credenciales de la app de Shopify, BD y `mensajero.base_url`.
4. **Servidor web:** - Apache: el `.htaccess` ya reescribe al front controller. - nginx (Hestia): aplica una plantilla con `try_files $uri $uri/ /index.php?$args;` (igual que hicimos para mensajero.gmartos.es).
5. Comprueba: `https://shopify.gmartos.es/health` → `{"status":"ok"}`.

App en el Partner Dashboard

1. Crea la app (distribución **Custom** para el Modelo A).
2. Copia **API key** y **API secret** a `config.php`.
3. App URL: `https://shopify.gmartos.es/` · Redirect: `https://shopify.gmartos.es/auth/callback`.
4. Instala en tu development store abriendo: `https://shopify.gmartos.es/install?shop=tu-tienda.myshopify.com`.

Flujo implementado

- `/` landing (presentación) · `/install` + `/auth/callback` (OAuth) · `/webhooks` (recibe y verifica HMAC) · `/app` (ajustes: pegar API key de Mensajero y plantilla) · `/health`.
- Webhook `orders/create` → extrae teléfono/idioma/productos → `POST mensajero/api/v1/messages`.
- GDPR: `customers/redact`, `customers/data_request`, `shop/redact`, `app/uninstalled`.

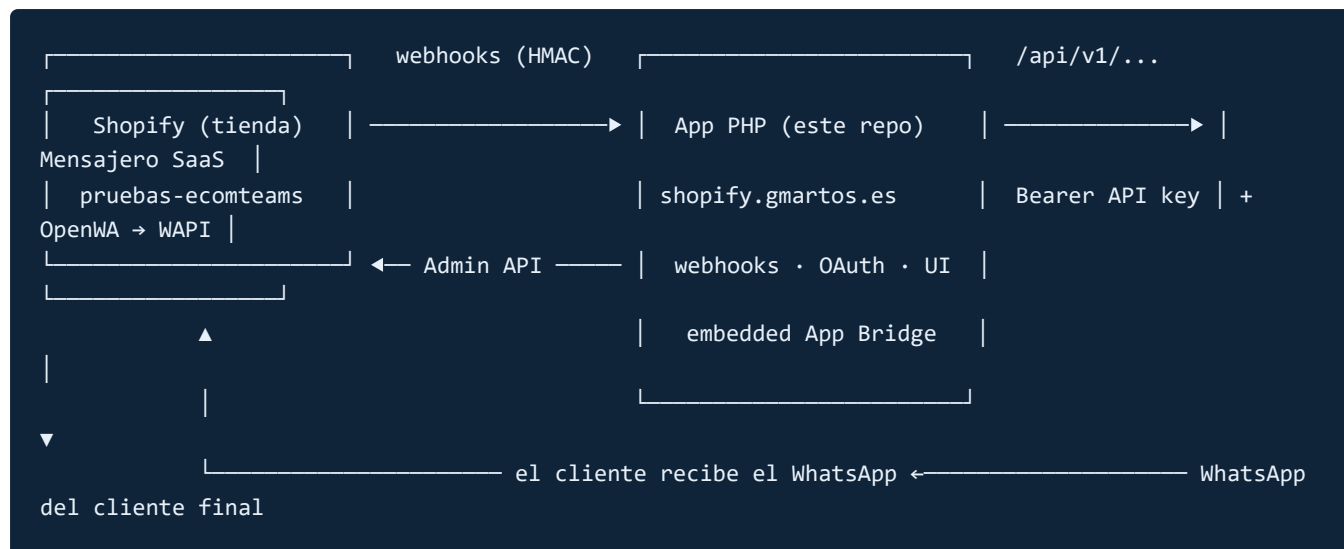
Pendiente (siguientes fases)

- Resto de avisos (envío/tracking, reembolso, bienvenida) y plantillas multiidioma.
- Idempotencia con `webhook_seen` (tabla ya creada).
- UI embebida con App Bridge/Polaris (ahora es una página simple).
- (Opcional) Modelo B: Billing API + provisión del plan en Mensajero.

Mensajero Shopify — Documentación técnica

App Shopify (PHP nativo, sin Composer) que conecta una tienda Shopify con el SaaS Mensajero (mensajero.gmartos.es) para enviar avisos por WhatsApp a los clientes finales de la tienda.

1. Arquitectura general



Tres capas:

1. **Shopify** – emite webhooks (`orders/create` , `fulfillments/create` , `refunds/create` , `customers/create` , `app/uninstalled` , GDPR) y expone Admin API para leer pedidos.
2. **App PHP** (este proyecto) – se aloja en `https://shopify.gmartos.es` , recibe los webhooks, traduce el evento a un mensaje WhatsApp y llama al panel Mensajero. Solo guarda: dominio de la tienda, access token de Shopify (cifrado) y la API key del cliente de Mensajero (cifrada).
3. **Mensajero SaaS** (panel `mensajero.gmartos.es`) – cuenta del cliente, plan, cuota, sesión WhatsApp (OpenWA), logs, cifrado, RGPD. Es el que entrega el mensaje vía OpenWA.

La app **no es multi-tenant en sí**: cada tienda instala la app, da su API key del panel Mensajero y se vincula al usuario correspondiente del SaaS. La cuenta del cliente vive en el panel, no en la app.

2. Componentes y ficheros

```

mensajero-shopify/
├── public/
│   ├── index.php          # Front controller (dispatch a Router)
│   └── .htaccess          # Rewrites a index.php
├── bootstrap.php          # Carga config, autoloader PSR-4, sesión
├── routes.php             # Definición de TODAS las rutas
├── app/
│   ├── helpers.php        # config(), e(), view(), redirect(), logline()
│   ├── Router.php         # Mini-router GET/POST + placeholders
│   ├── Shopify.php        # OAuth, HMAC webhooks, session token JWT, Admin API
│   ├── Db.php             # PDO + cifrado/descifrado de tokens al persistir
│   ├── Crypto.php         # AES-256-GCM (clave en config/security.key)
│   ├── Mensajero.php      # Cliente del panel Mensajero (/api/v1/*)
│   └── Events.php         # Definición de eventos: topic, label, vars, default
├── views/
│   ├── home.php           # Landing pública (sin shop=)
│   └── app.php            # UI embebida con pestañas
├── config/config.sample.php # Plantilla; el real config.php NO se sube a git
├── db/schema.sql          # 3 tablas: shops, shop_config, webhook_seen
└── docs/                 # Esta documentación

```

3. Modelo de datos (3 tablas)

```

shops          (shop_domain PK, access_token, created_at)
shop_config    (shop_domain PK, mensajero_api_key, events_json, updated_at)
webhook_seen   (id PK, topic, hmac, seen_at, KEY (topic, hmac)) -- dedupe

```

`events_json` guarda en JSON la config de la tienda:

```

{
  "new_order": {"on": 1, "tpl": "Hola {firstname}...", "i18n": {"es": "...", "en": "..."}},
  "shipped":   {"on": 0, "tpl": "Tu pedido {order}...", "i18n": {}},
  "welcome":   {"on": 0, "tpl": "...", "i18n": {}},
  "refund":    {"on": 0, "tpl": "...", "i18n": {}},
  "__filter": {"mode": 0, "list": "amazon,temu"}
}

```

4. Flujos completos

4.1 Instalación (OAuth)

1. El comerciante abre el enlace de instalación → Shopify redirige a `https://shopify.gmartos.es/?shop=<tienda>.myshopify.com`.
2. La app comprueba si la tienda ya está instalada; si no, redirige a `/install?shop=...`.

3. `/install` genera un `state` aleatorio, lo guarda en sesión y redirige a `https://<shop>/admin/oauth/authorize?...`.
4. Shopify pide permisos al comerciante y, al aceptar, vuelve a `/auth/callback?code=...&hmac=...&state=...`.
5. La app **verifica el HMAC** de la URL y el `state` de sesión.
6. Llama a `POST https://<shop>/admin/oauth/access_token` con `code` para canjear por un **access token permanente**.
7. Guarda el token en BD **cifrado** (`Db::saveShop` → `Crypto::encrypt`).
8. Redirige al admin de Shopify embebido (`https://<shop>/admin/apps/<api_key>`).

Importante: los webhooks **NO se registran aquí**. Están declarados en `shopify.app.toml` y Shopify los crea automáticamente al instalar. Esto se debe a que `use_legacy_install_flow = false` y los webhooks tipo PII requieren ser app-specific.

4.2 Webhook entrante (pedido nuevo)

1. Shopify hace POST a `/webhooks` con cabecera `X-Shopify-Hmac-Sha256`.
2. La app **verifica el HMAC** (`Shopify::verifyWebhook($raw, $hmac)` con el `api_secret`).
3. Responde 200 inmediatamente (Shopify exige <5s).
4. Procesa: identifica `topic` y `shop_domain`.
5. GDPR/uninstall: `app/uninstalled` y `shop/redact` borran la tienda. `customers/data_request` y `customers/redact` se registran en log (no almacenamos datos personales propios).
6. Eventos normales: lee `shop_config`, decodifica `events_json`, encuentra el evento según el topic (`Events::keyByTopic`), comprueba si está activado (`on: 1`).
7. Llama a `wh_vars($event, $data, $shop, $token)` que devuelve `[teléfono, variables, pedido]`. Para `shipped / refund` hace una llamada a Admin API para obtener el pedido completo (el webhook trae solo IDs).
8. Aplica filtro de forma de pago si corresponde (`wh_payment_allowed`).
9. Elige la plantilla por idioma del cliente (`customer_locale` → 2 letras).
10. Sustituye `{firstname}`, `{order}`, etc., y llama `Mensajero::send($apiKey, $phone, $text, $event)`.

4.3 UI embebida (App Bridge)

1. El comerciante abre la app desde Apps → mensajero_whatshap. Shopify carga `https://shopify.gmartos.es/app?shop=...&id_token=...&host=...`.
2. La app valida el `id_token` (JWT HS256 firmado con `api_secret`, ver `Shopify::verifySessionToken`).
3. Renderiza `views/app.php` con la config actual (decodificada con cifrado AES-GCM).
4. El navegador carga `cdn.shopify.com/shopifycloud/app-bridge.js` que expone `shopify.idToken()`.
5. Cada acción del usuario (guardar, enviar prueba, autorizar IP, ver estadísticas) hace `fetch` con `Authorization: Bearer <session_token>`.
6. El backend verifica el session token en cada llamada (no usa cookies).

4.4 Acciones autenticadas por session token

Ruta	Método	Acción
<code>/app/save</code>	POST	Guarda API key + eventos + filtro
<code>/app/test</code>	POST	Envía un WhatsApp de prueba al teléfono indicado
<code>/app/allowlist</code>	POST	Autoriza la IP de este servidor en la cuenta Mensajero
<code>/app/stats</code>	GET	Devuelve <code>/api/v1/logs</code> del panel para la tarjeta de estadísticas
<code>/app/dpa-status</code>	GET	Comprueba si el comerciante ha firmado el DPA + pinguea install
<code>/app/debug</code>	GET	Lista webhooks registrados + tail del log
<code>/app/rewebhooks</code>	POST	Re-registra webhooks vía REST (legacy fallback)

5. Seguridad

- **OAuth HMAC** verificado en `/auth/callback` con `hash_equals`.
- **Webhooks HMAC** verificados en `/webhooks` con `hash_equals`.
- **Session token JWT** validado (header+payload firmados con `api_secret`, comprobación de `exp` / `nbf` / `aud` / `dest`).
- **Cifrado en reposo AES-256-GCM** de `access_token` y `mensajero_api_key` (clave en `config/security.key`).
- **Sin cookies de autenticación:** la UI usa exclusivamente session tokens para sus llamadas.
- **Sin secretos en el cliente:** la API key del panel solo viaja del navegador al backend (al guardar) y luego del backend a Mensajero (con Bearer); nunca se devuelve al frontend.

6. Dependencias

- PHP 7.4+ (recomendado 8.x) con `openssl` y `curl`.
- MySQL 5.7+ / MariaDB 10.3+.
- HTTPS válido (Shopify exige TLS).
- Shopify CLI 4+ solo para gestionar la configuración (no necesario en runtime).

7. Variables de entorno (config/config.php)

```
return [  
    'env'      => 'prod',  
    'app_url'  => 'https://shopify.gmartos.es',  
    'shopify' => [  
        'api_key'    => '...',  
        'api_secret' => '...',  
        'scopes'     => 'read_customers,read_fulfillments,read_orders',  
        'webhooks'   => ['orders/create', 'fulfillments/create', 'refunds/create',  
        'customers/create', 'app/uninstalled'],  
    ],  
    'db'      =>  
    ['host'=>'127.0.0.1','port'=>3306,'name'=>'...','user'=>'...','pass'=>'...','charset'=>'utf8mb4']  
    ,  
    'mensajero'=> ['base_url' => 'https://mensajero.gmartos.es'],  
    'security' => ['key' => '<64 hex aleatorios>'],  
];
```

Mensajero Shopify — Guía de desarrollo

Cómo extender, depurar y desplegar la app.

1. Setup local

1. Clona o copia el repo en una carpeta servida por HTTPS (vía Hestia, nginx, lo que sea).
2. Copia `config/config.sample.php` → `config/config.php` y rellénalo.
3. Importa `db/schema.sql` en una BD vacía.
4. La app necesita ser accesible públicamente para que Shopify pueda llamar a `/webhooks` y `/auth/callback`. Para desarrollo local usa `ngrok` o un dominio real.

2. Despliegue de configuración (Shopify CLI)

La configuración de la app (scopes, webhooks subscriptions, URLs) **vive en** `shopify.app.toml`, no en el dashboard. Mantén una carpeta separada para esto:

```
z:/mensajero-shopify-config/  
└─ shopify.app.toml
```

Comandos:

```
# Enlazar con la app existente del Dev Dashboard  
shopify app config link --client-id=<TU_CLIENT_ID>  
  
# Editar shopify.app.toml localmente y publicar nueva versión:  
shopify app deploy
```

Cada `deploy` crea una versión nueva en el Dev Dashboard y la publica para todas las instalaciones.

3. Estructura de código

`app/Router.php`

Mini-router. Soporta `{placeholder}` en la URL. El handler puede ser un closure o `Controller@metodo`. Cuando hay placeholders los pasa al closure como array.

`app/Shopify.php`

Toda la integración con Shopify: HMAC, OAuth, JWT, Admin API.

Métodos clave: - `validShop($shop)` — anti-inyección, valida `*.myshopify.com`. - `verifyOAuthHmac($query)` — firma del callback OAuth. - `verifyWebhook($raw, $hmac)` — firma de webhook (base64). -

`verifySessionToken($jwt)` — devuelve dominio de la tienda si el JWT es válido. - `exchangeToken($shop, $code)` — code → access_token. - `getOrder($shop, $token, $orderId)` — Admin API REST.

`app/Events.php`

Single source of truth para la definición de los eventos disponibles.

Para añadir un evento nuevo:

```
public static function all(): array {
    return [
        'new_order' => [...],
        'shipped'   => [...],
        // tu_evento_nuevo:
        'cart_abandoned' => [
            'topic'   => 'checkouts/create',      // topic Shopify
            'label'   => 'Carrito abandonado',
            'vars'    => '{firstname} {checkout_url}',
            'default' => 'Hola {firstname}, has dejado un carrito sin terminar...',
        ],
    ];
}
```

Luego en `routes.php` → `wh_vars()` añade el case para construir teléfono/vars desde el payload del nuevo topic, y en `shopify.app.toml` añade el topic a la suscripción.

`app/Mensajero.php`

Cliente del panel SaaS. Cada método encapsula una llamada `/api/v1/*` con Bearer: - `send()` – mensaje de texto. - `sendMedia()` – adjunto (no usado actualmente desde la app Shopify). - `logs($apiKey, $limit)` – estadísticas para la tarjeta. - `dpaStatus($apiKey)` – si el DPA está firmado. - `pingInstall($apiKey, $siteUrl, $version)` – registra plataforma + versión. - `registerSelfIp($apiKey)` – autoriza la IP del servidor en la cuenta.

`app/Db.php`

PDO con métodos estáticos (`first` , `run` , helpers `saveShop` / `saveConfig` / `getConfig`).

Importante: `saveShop()` cifra el `access_token` con `Crypto::encrypt()` . `getShop()` lo descifra. Igual con `mensajero_api_key` en `saveConfig` / `getConfig` .

`app/Crypto.php`

AES-256-GCM. Formato `gcm:base64(iv|tag|ct)` . Soporta también descifrado `enc:` (legacy CBC) por compatibilidad.

`routes.php`

Aquí están todos los handlers. Es deliberadamente plano (closures), no MVC pesado.

4. Depuración

Log de la app

`storage/logs/app.log` — cada webhook procesado, cada error. Para verlo desde el navegador:

App embebida →  **Ajustes** → **Diagnóstico** → **Comprobar webhooks y log**.

Logs PHP

`storage/logs/php-error.log` — errores fatales (configurable en `bootstrap.php`).

Comprobaciones útiles

- ¿Llega el webhook? Mira el log tras hacer la acción en Shopify.
- ¿Está activo el evento? Comprueba `events_json` en BD: `SELECT events_json FROM shop_config WHERE shop_domain='...'`.
- ¿La firma falla? Verifica que `api_secret` en `config.php` coincide con el del Dev Dashboard.

5. Añadir una pestaña a la UI

`views/app.php` tiene un sistema de pestañas declarativo:

```
<nav class="nav">
  <button class="nav-tab" data-panel="stats">  Estadísticas</button>
  ...
</nav>

<section class="panel" data-panel="stats">...</section>
```

Para añadir una pestaña: 1. Añade un `<button>` con `data-panel="mi-tab"` en la nav. 2. Añade un `<section class="panel" data-panel="mi-tab" hidden>` con el contenido. 3. El JS de switching ya las muestra/oculta automáticamente.

6. Migraciones de BD

Hay 3 tablas (`shops` , `shop_config` , `webhook_seen`). Si añades campos:

1. Edita `db/schema.sql` con la columna nueva.
2. En producción, ALTER TABLE manualmente. La app no tiene "Reparar" propio (eso es del panel Mensajero).

7. Compatibilidad PHP

- Sin Composer.
- Solo funciones nativas (`openssl_*` , `curl_*` , `json_*` , `hash_*`).

- Compatible con PHP 7.4+. En 8.x los `curl_close()` salen como deprecated en el linter pero no son errores.

8. Pruebas manuales

Cada release, antes de subir a producción, verificar:

1. **Instalación limpia:** instalar en un dev store nuevo, comprobar que se crea la tienda y los webhooks.
2. **Crear pedido:** WhatsApp llega.
3. **Marcar fulfilled:** WhatsApp llega (si shipped está activado).
4. **Enviar prueba:** el botón funciona.
5. **Cambiar API key + Guardar:** la nueva se cifra (mira `mensajero_api_key` en BD, debe empezar por `gcm:`).
6. **Autorizar IP:** muestra IP.
7. **Estadísticas:** cargan al abrir.
8. **Diagnóstico:** muestra webhooks registrados.

9. Comandos útiles

```
# Lint PHP
php -l routes.php

# Ver log en vivo
tail -f storage/logs/app.log

# Generar clave segura para config security.key
php -r "echo bin2hex(random_bytes(32));"
```

Mensajero Shopify — Plan para pasar a comercial

Estado actual: la app es **custom distribution** atada a tu dev store. Para vender a clientes reales hay dos vías. Este documento explica las opciones, el trabajo necesario en cada una y mi recomendación.

1. Resumen ejecutivo

Vía	Para quién	Comisión Shopify	Revisión	Tiempo	Visibilidad
Modelo A — App Pública UNLISTED	Vender por enlace propio sin App Store	0% (tú facturas en Mensajero)	Sí, pero más laxa	1-2 semanas	No aparece en búsquedas; instalable solo por enlace
Modelo B — App Pública LISTED (App Store)	Captar tráfico orgánico de Shopify App Store	15-20% sobre el cobro	Sí, completa	2-6 semanas	Aparece en App Store, ranking, reseñas

Mi recomendación: Modelo A (Unlisted), porque encaja con tu modelo de negocio (cliente paga en mensajero.gmartos.es), evitas la comisión y la revisión es más sencilla. Si más adelante quieres visibilidad orgánica, una segunda app puede ir al App Store reutilizando el mismo código.

2. Por qué necesitas una app NUEVA (no esta)

La app actual `mensajero_whatsapp` está configurada como **Custom Distribution**, lo que la ata a una sola tienda. Shopify **no permite cambiar el método de distribución una vez elegido**. Por tanto, para producción hay que **crear una segunda app** en el Dev Dashboard con distribución pública.

Buena noticia: el **mismo código PHP** del repo sirve para ambas. Solo cambian las credenciales en `config/config.php`:

```
'shopify' => [  
    'api_key'    => '<NUEVO_client_id>',    // de la app pública  
    'api_secret' => '<NUEVO_client_secret>', // de la app pública  
    ...  
]
```

Puedes mantener las dos apps en paralelo (la dev para pruebas, la pública para clientes reales) apuntando a `config/config.php` distintos según el dominio.

3. Cambios técnicos necesarios en el código

3.1 Token Exchange (OBLIGATORIO en modelo público)

La app actual usa `use_legacy_install_flow = true`. Para distribución pública Shopify **exige el flujo moderno**, que se llama **Token Exchange**.

Qué cambia: en vez de OAuth clásico (redirect → code → exchange), Shopify carga la app embebida con un `id_token` (session token JWT) y la app lo cambia por el `access_token` mediante una llamada a `/admin/oauth/access_token` con `grant_type=token_exchange`.

Trabajo estimado: ~50 líneas de código nuevas en `Shopify.php` y un cambio menor en `routes.php`. El esqueleto:

```
// app/Shopify.php (a añadir):
public static function tokenExchange(string $shop, string $sessionToken, string $type =
'offline'): ?string {
    $r = self::http('POST', 'https://' . $shop . '/admin/oauth/access_token', [
        'client_id'          => config('shopify.api_key'),
        'client_secret'      => config('shopify.api_secret'),
        'grant_type'         => 'urn:ietf:params:oauth:grant-type:token-exchange',
        'subject_token'      => $sessionToken,
        'subject_token_type' => 'urn:ietf:params:oauth:token-type:id_token',
        'requested_token_type' => $type === 'online'
            ? 'urn:shopify:params:oauth:token-type:online-access-token'
            : 'urn:shopify:params:oauth:token-type:offline-access-token',
    ]);
    return json_decode($r['body'], true)['access_token'] ?? null;
}

// routes.php (en /app, antes de servir la UI):
// Si no tenemos shop instalada y llega un id_token válido → token exchange y guardar.
if (!Db::getShop($shop) && $sessionToken) {
    $token = Shopify::tokenExchange($shop, $sessionToken);
    if ($token) { Db::saveShop($shop, $token); }
}
```

Y en `shopify.app.toml` de la app nueva quitas `use_legacy_install_flow` (lo dejas implícito, false).

3.2 Mantener OAuth clásico como fallback (opcional)

Para instalaciones legacy o para clientes con custom apps internos podemos mantener `/auth/callback`. Recomendando dejarlo para no romper instalaciones existentes; coexiste sin problemas.

3.3 Pantalla de elección de plan tras la instalación (opcional)

Cuando alguien instala la app pública, lo natural es enviarle a una pantalla de "Conecta tu cuenta de Mensajero" o "Crea tu cuenta". Hoy hace falta que ya tenga API key. Mejora UX:

- Si al abrir la app no hay API key, mostrar pantalla con CTA "Crear cuenta en Mensajero" → `mensajero.gmartos.es/registro?return=<shop>`.

- Tras registro y pago, redirigimos de vuelta a la app con la API key auto-rellenada (parámetro firmado).

Esto es opcional pero mejora conversión.

4. Trabajo no técnico (Partner Dashboard / Dev Dashboard)

1. **Crear nueva app** en `dev.shopify.com` → "Create app".
2. **Configurarla:** - App URL: `https://shopify.gmartos.es/` - Allowed redirection URLs: `https://shopify.gmartos.es/auth/callback` - Scopes: `read_customers,read_fulfillments,read_orders` - Embedded: sí
3. **Distribución:** elige **Public distribution** (no App Store) → **Unlisted**.
4. **Solicitar Protected customer data** con motivos (los textos están en `docs/DOC-TECNICO.md`).
5. **Configurar webhooks** vía `shopify.app.toml` (los topics + privacy_compliance, igual que en la dev).
6. `shopify app deploy` → crea v1.0.0 de la app pública.

5. Revisión de Shopify

Aunque sea unlisted, Shopify revisa:

- **Webhooks de privacidad:** `customers/data_request`, `customers/redact`, `shop/redact` deben responder 200 y manejar correctamente. Ya lo hacen.
- **Política de privacidad** pública con la información del tratamiento. Apunta a `mensajero.gmartos.es/legal/privacidad` (ya redactada).
- **Página de instalación** o de inicio de la app. La home (`views/home.php`) ya está; ajustarla un poco antes de revisión (capturas, qué hace, planes).
- **Soporte:** una dirección de email visible (ya tienes soporte@gmartos.es).
- **Términos:** enlazar `mensajero.gmartos.es/legal/condiciones`.

Tiempo típico de aprobación unlisted: **3-10 días hábiles**.

6. Facturación

Opción A (recomendada): cobras tú en Mensajero

- El comerciante se registra en `mensajero.gmartos.es`, paga su plan (PayPal/Redsys, ya integrado).
- Recibe su API key, instala la app gratis (gratis para Shopify), pega la API key, listo.
- Tú te quedas el 100%.
- Único requisito de Shopify para apps unlisted: la app **no debe cobrar nada vía la app embebida** (no hay paywall en Shopify).

Opción B: Shopify Billing API

- Cobras vía Shopify y se queda el 15-20%.

- Tendrías que añadir código de `RecurringApplicationCharge` y flujo de aprobación.
- Solo tiene sentido si vas a App Store y quieres "1-click install + pay".

Para tu caso quédate con A.

7. Soporte multi-tienda por cliente

Un mismo comerciante puede tener varias tiendas Shopify. Hoy cada instalación apunta a una API key del cliente, y `installations` registra cada par (`user_id` , `platform` , `site_url`). El cliente puede usar **la misma API key** en varias tiendas — el sistema de allowlist por IP funciona; cada tienda hace ping desde su servidor y se autoriza. No hace falta cambio de código.

8. Roadmap concreto

Semana 1

1. Crear la app pública nueva en Dev Dashboard.
2. Configurar `shopify.app.toml` (nueva carpeta).
3. Implementar `tokenExchange` en `Shopify.php` .
4. `shopify app deploy` .
5. Probar instalación en otro dev store distinto del actual.

Semana 2

1. Capturas y descripciones para la página de la app.
2. Solicitar revisión a Shopify (unlisted).
3. Mientras esperan: pulir UX de "primera vez" (pantalla cuando no hay API key).

Tras aprobación

1. Generar enlace de instalación → compartirlo con primeros clientes.
2. Cobrar en `mensajero.gmartos.es` como ya hacemos.
3. Si quieres, repetir el proceso para listarla en App Store.

9. Riesgos y mitigación

Riesgo	Mitigación
Shopify rechaza por algún detalle de la revisión	Las apps unlisted suelen aprobarse al primer intento; corregir lo señalado y reenviar (rápido).
Token exchange falla con clientes antiguos	Mantener OAuth clásico como fallback evita romper migraciones.
Un cliente cancela su plan en Mensajero pero la app sigue instalada	El SaaS deja de aceptar envíos (allowlist + status <code>cancelled</code>). El cliente ve los errores en estadísticas. Idealmente, añadimos un banner cuando el plan no está activo.
Pico de tráfico al instalarse muchos	OpenWA escala mal por sesiones; presupuesta un servidor más grande o switch a Baileys (ya documentado en otros docs).

10. Coste estimado de esta migración

- **Código:** ~4-6 horas (token exchange + tests + pantalla UX).
- **Configuración Shopify:** ~1 hora.
- **Documentación de la página de la app + capturas:** 2-3 horas.
- **Revisión Shopify:** tiempo de espera, no de trabajo.

Total estimado: ~1 día de trabajo efectivo, repartido en una semana real con la revisión.